

Relatório de Atividades: Desenvolvimento de API RESTful com FastAPI

Disciplina: Sistemas Distribuídos

Instituição: UFC - Campus Quixadá

Autor(a): Gabriely Correia Dealem

1. Introdução e Objetivo

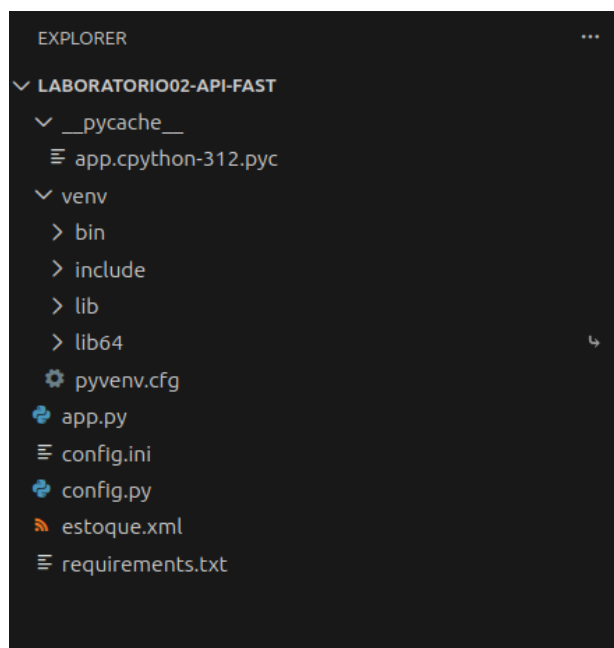
Esta atividade consistiu na criação de um serviço web seguindo o estilo arquitetural REST utilizando o framework **FastAPI**. O foco foi a implementação de operações CRUD (Create, Read, Update, Delete) para o gerenciamento de um inventário de aparelhos. O objetivo foi explorar a agilidade do ecossistema Python para sistemas distribuídos, garantindo a validação automática de dados e a documentação interativa.

2. Tecnologias e Ferramentas

- **Linguagem:** Python 3.12.
- **Framework Web:** FastAPI.
- **Servidor ASGI:** Uvicorn.
- **Validação de Dados:** Pydantic.
- **Ambiente Isolado:** Virtualenv (venv).

3. Arquitetura do Sistema

Abaixo, detalha-se a organização do projeto e o fluxo de dados entre os componentes.



3.1. Estrutura do Projeto (Explorer)

A aplicação utiliza uma estrutura organizada para facilitar a manutenção e o deploy:

- **app.py**: Ponto de entrada da aplicação e definição dos endpoints.

```
app.py > ...
1  from fastapi import FastAPI, HTTPException
2  from pydantic import BaseModel
3  import xml.etree.ElementTree as ET
4  import configparser
5
6  config = configparser.ConfigParser()
7  config.read('config.ini')
8  ARQUIVO = config['DEFAULT'].get('arquivo')
9  app = FastAPI()
10
11  class Aparelho(BaseModel):
12      id: int
13      nome: str
14      marca: str
15      preco: float
16      categoria: str
17      deposito: str
18
19  class Transferivel:
20      def transferir(self, novo_deposito):
21          self.deposito = novo_deposito
22
23  def carregar_aparelhos_xml():
24      try:
25          tree = ET.parse(ARQUIVO)
26          root = tree.getroot()
27          aparelhos = []
28          for aparelho in root.findall("aparelho"):
29              aparelhos.append({
30                  "id": int(aparelho.find("id").text),
31                  "nome": aparelho.find("nome").text,
32                  "marca": aparelho.find("marca").text,
33                  "preco": float(aparelho.find("preco").text),
34                  "categoria": aparelho.find("categoria").text,
35                  "deposito": aparelho.find("deposito").text,
36              })
37          return aparelhos
38      except FileNotFoundError:
39          root = ET.Element("estoque")
40          tree = ET.ElementTree(root)
41          tree.write(ARQUIVO, encoding="utf-8", xml_declaration=True)
42          return []
43
44  def salvar_aparelhos_xml(aparelhos):
45      root = ET.Element("estoque")
46      for aparelho in aparelhos:
47          aparelho_elem = ET.SubElement(root, "aparelho")
48          ET.SubElement(aparelho_elem, "id").text = str(aparelho["id"])
49          ET.SubElement(aparelho_elem, "nome").text = aparelho["nome"]
```

```

app.py  X  pyvenv.cfg  estoque.xml  config.py  config.ini
app.py > ...
44 def salvar_aparelhos_xml(aparelhos):
48     ET.SubElement(aparelho_elem, "id").text = str(aparelho["id"])
49     ET.SubElement(aparelho_elem, "nome").text = aparelho["nome"]
50     ET.SubElement(aparelho_elem, "marca").text = aparelho["marca"]
51     ET.SubElement(aparelho_elem, "preco").text = str(aparelho["preco"])
52     ET.SubElement(aparelho_elem, "categoria").text = aparelho["categoria"]
53     ET.SubElement(aparelho_elem, "deposito").text = aparelho["deposito"]
54
55     tree = ET.ElementTree(root)
56     tree.write(ARQUIVO, encoding="utf-8", xml_declaration=True)
57
58 @app.post("/aparelhos", status_code=201)
59 def adicionar_aparelho(aparelho: Aparelho):
60     aparelhos = carregar_aparelhos_xml()
61     if any(a["id"] == aparelho.id for a in aparelhos):
62         raise HTTPException(status_code=400, detail="ID já existe.")
63     aparelhos.append(aparelho.dict())
64     salvar_aparelhos_xml(aparelhos)
65     return {"erro": False, "message": "Aparelho adicionado com sucesso"}
66
67 @app.get("/aparelhos")
68 def listar_aparelhos():
69     return carregar_aparelhos_xml()
70
71 @app.get("/aparelhos/{id}")
72 def buscar_aparelho(id: int):
73     aparelhos = carregar_aparelhos_xml()
74     for aparelho in aparelhos:
75         if aparelho["id"] == id:
76             return aparelho
77     raise HTTPException(status_code=404, detail="Aparelho não encontrado.")
78
79 @app.put("/aparelhos/{id}")
80 def atualizar_aparelho(id: int, aparelho_atualizado: Aparelho):
81     aparelhos = carregar_aparelhos_xml()
82     for aparelho in aparelhos:
83         if aparelho["id"] == id:
84             aparelho.update(aparelho_atualizado.dict())
85             salvar_aparelhos_xml(aparelhos)
86             return {"erro": False, "message": "Aparelho atualizado com sucesso"}
87     raise HTTPException(status_code=404, detail="Aparelho não encontrado.")
88
89 @app.delete("/aparelhos/{id}")
90 def deletar_aparelho(id: int):
91     aparelhos = carregar_aparelhos_xml()
92     aparelhos_atualizados = [a for a in aparelhos if a["id"] != id]
93     if len(aparelhos) == len(aparelhos_atualizados):
94         raise HTTPException(status_code=404, detail="Aparelho não encontrado.")
95     salvar_aparelhos_xml(aparelhos_atualizados)

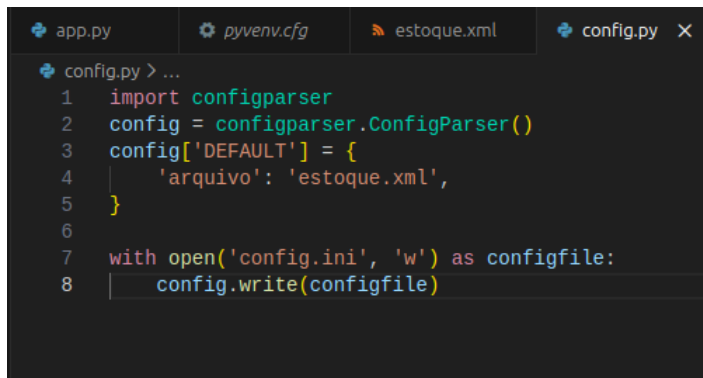
```

```

app.py  X  pyvenv.cfg  estoque.xml  config.py  config.ini
app.py > ...
90 def deletar_aparelho(id: int):
93     if len(aparelhos) == len(aparelhos_atualizados):
94         raise HTTPException(status_code=404, detail="Aparelho não encontrado.")
95     salvar_aparelhos_xml(aparelhos_atualizados)
96     return {"erro": False, "message": "Aparelho removido com sucesso"}
97
98 @app.put("/aparelhos/{id}/transferir/{novo_deposito}")
99 def transferir_aparelho(id: int, novo_deposito: str):
100     aparelhos = carregar_aparelhos_xml()
101     for aparelho in aparelhos:
102         if aparelho["id"] == id:
103             aparelho["deposito"] = novo_deposito
104             salvar_aparelhos_xml(aparelhos)
105             return {"erro": False, "message": "Aparelho transferido com sucesso"}
106     raise HTTPException(status_code=404, detail="Aparelho não encontrado.")

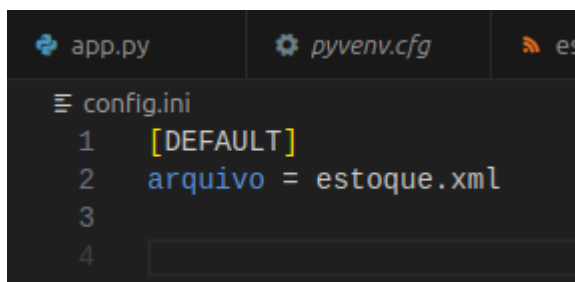
```

- **config.py** e **config.ini**: Gerenciamento centralizado de configurações do sistema.



The screenshot shows a code editor with four tabs: app.py, pyvenv.cfg, estoque.xml, and config.py. The config.py file is active and contains the following Python code:

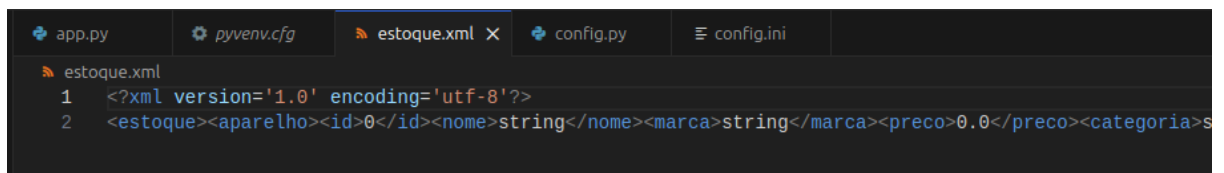
```
1 import configparser
2 config = configparser.ConfigParser()
3 config['DEFAULT'] = {
4     'arquivo': 'estoque.xml',
5 }
6
7 with open('config.ini', 'w') as configfile:
8     config.write(configfile)
```



The screenshot shows a code editor with three tabs: app.py, pyvenv.cfg, and estoque.xml. The config.ini file is active and contains the following INI file content:

```
1 [DEFAULT]
2 arquivo = estoque.xml
3
4
```

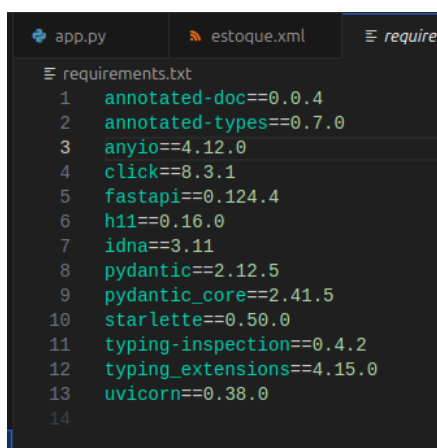
- **estoque.xml**: Fonte de dados ou arquivo de exportação para interoperabilidade.



The screenshot shows a code editor with five tabs: app.py, pyvenv.cfg, estoque.xml, config.py, and config.ini. The estoque.xml file is active and contains the following XML code:

```
1 <?xml version='1.0' encoding='utf-8'?>
2 <estoque><aparelho><id></id><nome>string</nome><marca>string</marca><preco>0.0</preco><categoria>s
```

- **requirements.txt**: Lista de dependências para reprodutibilidade do ambiente.



The screenshot shows a code editor with three tabs: app.py, estoque.xml, and requirements.txt. The requirements.txt file is active and contains the following list of dependencies:

```
1 annotated-doc==0.0.4
2 annotated-types==0.7.0
3 anyio==4.12.0
4 click==8.3.1
5 fastapi==0.124.4
6 h11==0.16.0
7 idna==3.11
8 pydantic==2.12.5
9 pydantic_core==2.41.5
10 starlette==0.50.0
11 typing-inspection==0.4.2
12 typing_extensions==4.15.0
13 uvicorn==0.38.0
14
```

- **venv/**: Ambiente virtual garantindo o isolamento do projeto.

```
app.py  estoque.xml  pyvenv.cfg  config.py  config.ini
venv > pyvenv.cfg
1  home = /usr/bin
2  include-system-site-packages = false
3  version = 3.12.3
4  executable = /usr/bin/python3.12
5  command = /home/gaby/Documentos/SD/Laboratorio02-API-Fast/venv/bin/python3 -m venv /home/gaby/Documentos/SD/Laboratorio02-API-Fast/venv
6
```

4. Implementação da Arquitetura

4.1. Definição do Modelo (Pydantic)

Utilizou-se a classe `BaseModel` do Pydantic para definir o contrato de dados do objeto "Aparelho". Isso garante que qualquer dado recebido via POST seja validado automaticamente quanto ao tipo (ex: `preco` como `float`).

4.2. Persistência em Memória

Para fins laboratoriais, os dados foram armazenados em uma lista Python (`db`), simulando o comportamento de um banco de dados enquanto o processo do servidor estiver ativo.

4.3. Interface de Exposição (Endpoints)

Foram implementados os seguintes métodos:

- **GET `/aparelhos`**: Lista todos os itens cadastrados.
- **POST `/aparelhos`**: Realiza o cadastro de novos aparelhos com validação de ID duplicado.
- **DELETE `/aparelhos/{id}`**: Remove um registro específico da memória.

FastAPI 0.1.0 OAS 3.1
/openapi.json

default		^
GET	/aparelhos	Listar Aparelhos
POST	/aparelhos	Adicionar Aparelho
GET	/aparelhos/{id}	Buscar Aparelho
PUT	/aparelhos/{id}	Atualizar Aparelho
DELETE	/aparelhos/{id}	Deletar Aparelho
PUT	/aparelhos/{id}/transferir/{novo_deposito}	Transferir Aparelho
Schemas		^

5. Procedimentos de Configuração e Execução

5.1. Superação de Desafios Técnicos

Durante a instalação, identificou-se o erro `externally-managed-environment` devido às políticas de segurança das distribuições Linux modernas. O problema foi mitigado através da criação e ativação de um ambiente virtual:

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install fastapi uvicorn
```

5.2. Execução do Servidor

O servidor foi inicializado em modo de desenvolvimento (hot-reload) através do comando:

```
python3 -m uvicorn app:app --reload
```

6. Resultados e Validação

A validação foi realizada através da interface **Swagger UI** (<http://127.0.0.1:8000/docs>), que é gerada automaticamente pelo FastAPI.

- **Teste de Sucesso:** O envio de um JSON via POST retornou o status code **201 (Created)**.
- **Teste de Restrição:** A tentativa de cadastrar um aparelho com um ID já existente disparou uma **HTTPException** com status **400 (Bad Request)** e a mensagem "**ID já existe**", conforme validado nos logs do sistema.

7. Conclusão

A prática demonstrou a eficiência do FastAPI em comparação com modelos tradicionais, destacando-se pela velocidade de desenvolvimento e robustez na validação de tipos. A arquitetura proposta permitiu uma separação clara entre a estrutura de dados (model) e as regras de acesso (routes), essencial para a construção de microsserviços em sistemas distribuídos modernos.